



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE307L_Compiler Design



Code Generation

**Dr. B.V. Baiju,
SCOPE,
Assistant Professor
VIT, Vellore**

Activation Record

- An activation record is a **contiguous block of storage** that manages information required by a single execution of a procedure.
- This information includes the **procedure's arguments**, **local variables**, and **return address**.
- **When you enter a procedure, you allocate an activation record, and when you exit that procedure, you de-allocate it.**
- Basically, it stores the status of the current activation function.
- So, whenever a function call occurs, then a new activation record is created and it will be pushed onto the top of the stack.
- It will remain in stack till the execution of that function.
- So, once the procedure is completed and it is returned to the calling function, this activation function will be popped out of the stack.
- Activation record is also known as **stack frames** or **function call frames** used by the compiler to manage the execution of a function or procedure.

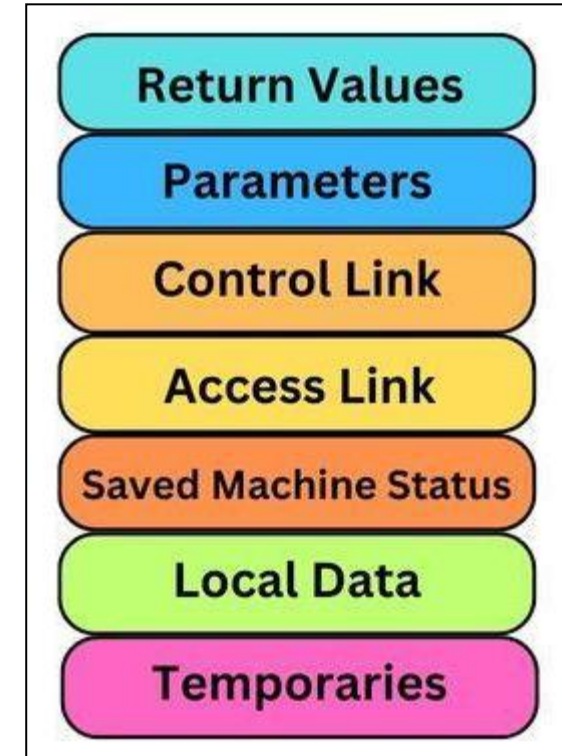
Components of Activation Records

1. Return Values

- This *address holds the location where the control should return* upon task completion.
- This feature helps the program continue executing from the same point it was initially created.
- It is used by a calling function that will return a value to the same calling function.

2. Parameters

- The calling procedure uses it to supply parameters to the called method.
- It stores actual parameters used to send input to the called system.
- The *parameters can be passed by value or reference* and stored in the activation record for the function to access.

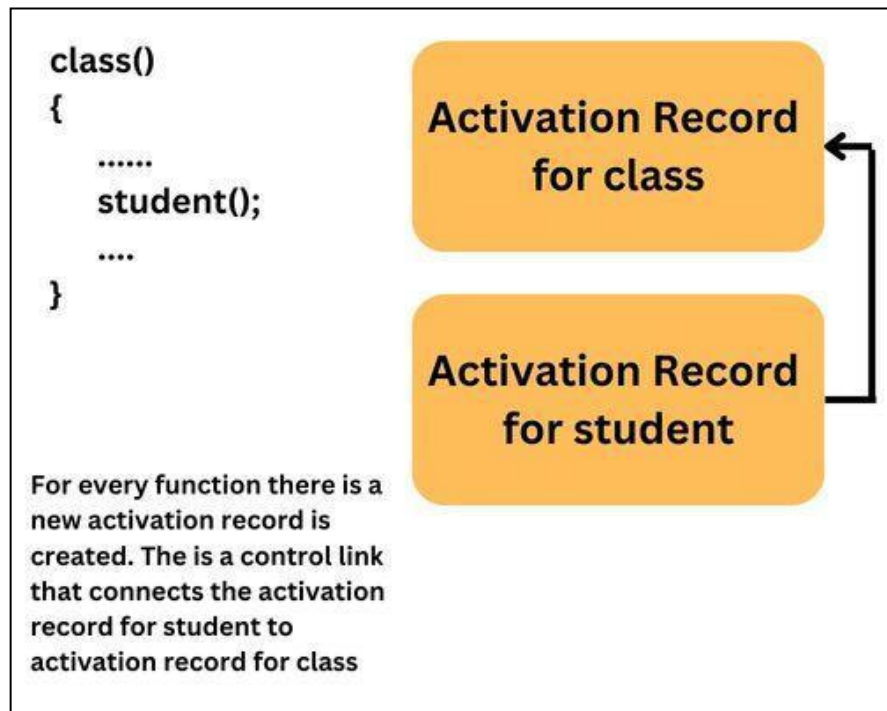
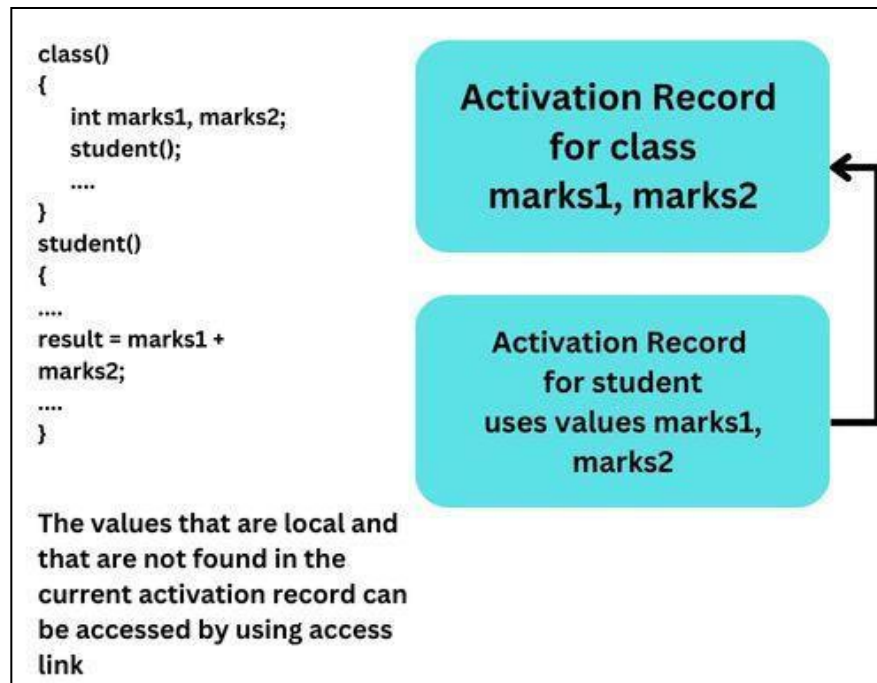


3. Control Link

- It points toward the activation record of the caller.
- It allows you to return and execute continuously.
- The system uses it to store information outside the local scope.
- The control link connects the activation record to the activation record of the caller.

4. Access Link

- It **stores the address of the activation record** of the caller function.



5. Saved Machine Status

- The activation record consists of critical information about the program's state, which is just about to get called.
- It stores information like the ***return address*** or ***machine registers***.
- The saved machine ensures the program can resume execution once the procedure call gets terminated.

6. Local Data

- This field consists of local data for a particular function's execution.
- Local data consists of variables that serve the purpose of quick calculations or storing specific values of a currently used function.

7. Temporaries

- It refers to the variables or storage locations used to ***store intermediate values within the procedure***.
- When a function executes, it may perform different operations that require temporary storage.
- Once the procedure call completes and the control returns to the calling code, the system deallocates activation records and releases temporaries.

```
#include<stdio.h>
void add();      //Function Declaration
int main()
{
    add();      //Function Calling
    return 0;
}
void add()      //Function Definition
{
    int a,b,c;  printf("\nEnter The Two values:");
    scanf("%d%d",&a,&b);
    c=a+b;
    printf("Addition:%d",c);
}
```

+-----+		
	Activation Record of main	
+-----+		
	Return Address: OS	<- Points to the OS or PF
+-----+		
	Control Link: OS	<- Points to the OS or PF
+-----+		
	Access Link: OS	<- Points to the OS or PF
+-----+		
	Saved Machine Status	<- Stores registers, etc.
+-----+		
	Local Data: None	
+-----+		
	Temporaries: None	
+-----+		

+-----+		
	Activation Record of add	
+-----+		
	Return Address: main	<- Points back to main()
+-----+		
	Control Link: main	<- Points back to main()
+-----+		
	Access Link: main	<- Points back to main()
+-----+		
	Saved Machine Status	<- Stores registers, etc.
+-----+		
	Parameters: None	<- No parameters in this function
+-----+		
	Local Data:	
	int a	
	int b	
	int c	
+-----+		
	Temporaries: None	
+-----+		

..

```
Printf ("Enter Your Name: ");  
Scanf ("%s", username);  
Show_data (username);  
printf ("Press any key to continue...");  
...  
int show_data (char *user)  
{  
Printf ("Your name is %s", username);  
return 0;  
}  
...
```

Activation tree

